

Chatbot Security & Cost Protection

How the public AI chat widget protects itself against abuse, floods and runaway costs

Project: `mcp_chatbot` (Odoo addon + FastAPI chat server) • Updated 12 June 2026

Why this matters in one sentence. The chat widget is open to every website visitor — even people who never log in — and every message they send is processed by a paid AI service, billed to the shop owner. Without protection, one person with a script could flood the bot, run up a large bill, or quietly use it as a free AI service for their own projects. The protections below make sure a real customer never notices anything, while an abuser hits a wall at every step.

The journey of one message (and its checkpoints)

When a visitor opens the website, the browser receives a short-lived **access pass** (a signed token). Every message must carry this pass. From there, each message walks through a series of checkpoints **before** any money is spent on AI processing:

- **Checkpoint 1 — valid pass?** No valid token, no service. The pass expires after one hour and is renewed automatically for genuine visitors.
- **Checkpoint 2 — is the bot switched on?** The shop admin has an online/offline switch; "offline" is enforced on the server, not just hidden in the page.
- **Checkpoint 3 — reasonable size?** Messages over 2,000 characters are rejected. Nobody can paste a whole book and make the shop pay to process it.
- **Checkpoint 4 — reasonable speed?** At most 1 message every 2 seconds and 20 per minute, per person. A human never notices; a flooding script is stopped at its first burst.
- **Checkpoint 5 — daily allowance left?** Each person has a daily AI usage budget. When it is spent, only *that person* waits until tomorrow — everyone else chats normally.

Only a message that passes all five checkpoints reaches the AI. Rejected messages cost **nothing**.

Tier 0 — Protections built into our own code

IMPLEMENTED**1 · Message size limit****THE RISK**

The AI bills by the amount of text it reads. A single message containing a pasted document could cost as much as hundreds of normal questions.

WHAT WE BUILT

A hard cap of 2,000 characters per message, enforced twice: the chat box itself stops accepting text at the limit (so honest users get instant feedback), and the server rejects oversized messages anyway (so the limit can't be bypassed by a script talking to the server directly).

The coffee machine pours cups, not buckets.

IMPLEMENTED**2 · Speed limit (rate limiting)****THE RISK**

A simple script can send hundreds of messages per minute — each one a paid AI call. A human conversation never looks like that.

WHAT WE BUILT

Two speed ceilings on the server, applied per person: at most **1 message every 2 seconds** (stops rapid-fire bursts) and **20 messages per minute** (caps sustained volume). Logged-in users are identified by their account; visitors are identified by their network address. Tripping a ceiling returns a polite “you're sending messages too quickly” notice in the chat — and the blocked requests cost nothing.

One cup at a time; the queue moves at human speed.

THE RISK

Even at polite speed, someone chatting all day — or using the bot as a free AI service — can burn through real money. And a single shared “kill switch” would be worse: one abuser reaching it would shut the bot down for every genuine customer too.

WHAT WE BUILT

Every person gets their **own** daily allowance of AI usage, measured in the same units the AI provider bills (tokens). When one person spends theirs, only they are paused until the next day — nobody else is affected.

- **Logged-in customers:** 500,000 tokens per day (roughly several hundred long answers — far more than any real customer uses).
- **Anonymous visitors:** 300,000 tokens per day, counted per network address.
- **Identity upgrade bonus:** a visitor who verifies their email in the chat (for example to place an order) earns an extra 200,000 — same level as a logged-in customer.
- **Checkout protection:** the moment a visitor starts email verification, a small extra grace (30,000) is granted — so someone about to complete an order is never cut off mid-purchase. A sale is never lost to the limit.

The experience is designed to feel fair, never sudden:

- At **90%** of the allowance, a gentle amber notice appears once: *“You’re approaching today’s usage limit.”*
- An answer that is already being written is **never cut off mid-sentence** — the visitor always gets their reply; the pause only applies from the next message.
- When the allowance is spent, a clear notice explains it: *“You’ve reached today’s usage limit. Please try again tomorrow.”*

For the shop admin, everything is visible and adjustable in Odoo Settings: a master on/off switch for the limits, the budget amounts, the bonus and the grace. Every person’s daily consumption is recorded and browsable in the backend (Token Usage records) — and the recording continues even when limits are switched off, so decisions are always based on real numbers.

Every customer has their own daily coffee card; when one card is empty, the machine keeps serving everyone else.

IMPLEMENTED

4 · Master switch, enforced for real

THE RISK

An “offline” switch that only hides the chat window is decoration: a script talking directly to the server would still be answered — and billed.

WHAT WE BUILT

The online/offline status is enforced **on the server**. When the bot is offline, no session is created, nothing is stored, and no AI call is made — regardless of how the request arrives. The widget also shows an Offline badge so genuine visitors aren't confused.

Unplugging the machine, not just hanging a "closed" sign on it.

IMPLEMENTED

5 · Conversations get a shelf life (privacy)

THE RISK

Chat transcripts contain personal information — names, emails, order details. Keeping them forever grows the legal exposure (privacy laws such as GDPR require a defined retention period) and means a database breach would leak the shop's entire chat history since launch.

WHAT WE BUILT

An automatic daily cleanup deletes closed conversations older than **90 days** (configurable in Settings). What the bot has learned about each customer — preferences, allergies, profession — is kept separately as short, editable facts, so the bot stays personal while the raw transcripts expire.

The barista remembers you take oat milk — not every word of every chat you've ever had.

Tier 1 — A bouncer at the edge (Cloudflare)

THE RISK — TWO PROBLEMS TIER 0 CANNOT SOLVE ALONE

(a) All the checkpoints above live *inside* our server. To say “no” to a request, the server must first receive it — spending connections, processing power and bandwidth. A large enough automated flood can overwhelm the machine even though every request is being rejected.

(b) The per-visitor limits identify anonymous visitors by their network address (IP). But on the open internet, the header that carries that address is just text the sender can type. An attacker talking to the server directly could fake a fresh address on every request — appearing as thousands of “new visitors” and slipping past the per-visitor limits.

WHAT WE BUILT

Cloudflare (free plan) is placed in front of the server. All traffic hits Cloudflare first: known bots, attack scripts and floods are dropped on *Cloudflare's* machines before they ever reach — or cost — anything on ours. Genuine visitors pass through and notice nothing.

It also fixes the identity problem: Cloudflare *observes* each visitor's real address at the network level and stamps it onto the request — a measurement, not a claim. Our server has been updated to **only believe address information that arrives from machines we trust** (our own proxy, Cloudflare); anything a client types itself is ignored. Faked-address tricks are verified blocked by automated tests.

- **In the code (done):** trusted-proxy address resolution, Cloudflare header support, fail-safe defaults.
- **At deployment (a ~30-minute checklist, see `docs/cloudflare_setup.md`):** point the domain at Cloudflare, enable bot protection and an edge speed limit, and lock the server's firewall so traffic *cannot* go around the bouncer.

A bouncer on the sidewalk stops troublemakers before they touch the machine — and he's the only one allowed to write names on the guest list.

Planned next (added only if real abuse appears)

7 · A quiet “prove you're human” check

Sophisticated bots can imitate real browsers. If abuse continues after Tiers 0 and 1, a modern invisible human-check (Cloudflare Turnstile) can be added before a stranger's first message — a real person sees nothing and just chats; a bot is silently stopped. Deliberately not added yet: no friction for real visitors before it's proven necessary.

PLANNED — LATER

8 · Content filtering on the bot's answers

Protection against visitors trying to trick the bot into saying something off-brand (“jailbreaking”). Real, but a reputation concern rather than a cost concern — scheduled after the money and flooding protections have proven themselves in production.

Summary at a glance

Protection	What it stops	Status
Access pass (signed token)	Anonymous scripts calling the chat server without going through the website	✓ Implemented
Message size limit (2,000 chars)	Paying to process pasted documents	✓ Implemented
Speed limit (1 / 2s, 20 / min)	Message floods from scripts	✓ Implemented
Personal daily budgets (500k / 300k tokens + bonuses)	One person running up the bill or freeloading on the AI — without ever blocking anyone else, and without ever losing a sale mid-checkout	✓ Implemented
90% early warning + fair blocking	Surprise cut-offs; answers interrupted mid-sentence	✓ Implemented
Server-enforced offline switch	The bot answering (and billing) while “switched off”	✓ Implemented
90-day conversation cleanup	Unbounded privacy/legal exposure; worst-case breach damage	✓ Implemented
Trusted address resolution	Faking network addresses to dodge the per-visitor limits	✓ Implemented
Cloudflare edge protection	Floods and bots reaching the server at all	● At deployment
Invisible human check (Turnstile)	Advanced bots imitating real browsers	● If needed
Answer content filtering	“Jailbreak” attempts against the brand voice	● Later

The design principle behind all of it: every limit targets *one person's* behaviour, never the whole service — an abuser gets stopped, a customer never notices, and a visitor in the middle of buying something is always allowed to finish.

Technical references: rate limiting & budgets enforced in the FastAPI chat server (`slowapi`), per-identity daily rows in `mcp.chatbot.usage`); budgets, bonuses and retention configurable in Odoo Settings → MCP Chatbot; Cloudflare deployment checklist in `docs/cloudflare_setup.md` . All protections verified by automated tests on the `add_security` branch.